Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas

Proyecto 2

Sonda de Kernel en C y Daemon en Go para la Telemetría de Contenedores.

PONDERACIÓN: 15pts

Tiempo estimado: 60hrs

Índice

1. Resumen Ejecutivo **3**
2. Competencia que desarrollaremos **3**
3. Objetivos del Aprendizaje **4**
3.1 Objetivo General 4
3.2 Objetivos Específicos **4**
4. Enunciado del Proyecto **5**
4.1 Descripción del problema a resolver **5**
MÓDULO DE KERNEL **6**
CRONJOB **7**
GRAFANA **8**
Daemon de GO **10**
1. Inicio del servicio 10
2. Cronjob 10
3. Ejecución del script para cargar el módulo de kernel 10
4. Loop principal (ejecución cada 20 segundos) 10
5. Finalización del servicio 10
RESTRICCIONES CLAVE **11**
Contenedores en la máquina 11
Análisis y ordenamiento de contenedores 11
Toma de decisiones 11
ARQUITECTURA 12
4.2 Alcance del proyecto **13**
4.4 Entregables 14
5. Metodología **16**
6. Desarrollo de Habilidades Blandas **17**
6.1 Proyectos Individuales **17**
7. Cronograma **18**
8. Rúbrica de Calificación **19**
8.1 Requisitos para optar a la calificación 19
8.2 Resumen de Puntuaciones **19**
8.3 Detalle de la Calificación **20**
8.4 Valores 22

1. MARCO FORMATIVO

1.1. Valor

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Responsabilidad	Mejorar y entregar nuestros trabajos como parte de las responsabilidad del curso

1.2. Competencia(s)

Indique la competencia general del curso y la competencia específica trabajada en esta lectura.

Tipo de Competencia	
Competencia General	Aprender el uso y funciones de un SO
Competencia Específica	Aprender Linux a fondo y las máquinas virtuales para su suo

1.3. Objetivo SMART

Define el/los objetivo(s) que se pretende cumplir con la tarea planteada, con un enfoque SMART, toma en consideración que un objetivo SMART, va enfocado a la habilidad(y no a tanto a la teoría

SMART	Definición	Objetivo redactado
Específico (¿Qué?)	El objetivo es concreto y tangible.	Los estudiantes serán capaces de configurar la monitorización de recursos usando un módulo Kernel en C, herramientas eBPF
Medible (¿Cuánto?)	El objetivo tiene una medida objetiva de éxito.	Lograr al menos un 80% de aciertos en las prácticas y evaluaciones de telemetría.
Alcanzable (¿Cómo?)	El objetivo debe ser posible con los recursos disponibles.	Utilizando código C, Go y las guías de eBPF proporcionadas.
Realista (¿Para qué?)	El objetivo contribuye a metas más amplias.	Equipar al estudiante con habilidades prácticas en observabilidad a nivel de kernel, preparándose para roles DevOps y SRE.
A Tiempo (¿Cuándo?)	El objetivo tiene fecha límite o mejor aún un cronograma de hitos de progreso	Cumplir este aprendizaje en el ciclo de duración del proyecto.

2. Resumen Ejecutivo

Este proyecto desarrollará un sistema integral compuesto por un módulo de kernel tradicional en C, una sonda de observabilidad eBPF, y una aplicación en GO (Daemon) diseñada para monitorizar, analizar y gestionar de manera autónoma contenedores en tiempo real. El sistema interactúa directamente con el núcleo del sistema operativo para recopilar datos sobre los contenedores (PID, consumo de recursos, llamadas al sistema) y tomar acciones proactivas, todo visualizado a través de Grafana.

3. Enunciado del Proyecto

Este proyecto tiene como objetivo principal diseñar e implementar un sistema integral para la monitorización proactiva, análisis automatizado y gestión inteligente de contenedores en entornos Linux.

4.1 Descripción del problema a resolver

En la administración de sistemas y el desarrollo de aplicaciones contenerizadas, obtener información detallada sobre los procesos en ejecución y tomar acciones automatizadas es un desafío crítico. Aunque herramientas como ps o docker stats ofrecen datos básicos, carecen de acceso directo a las estructuras del kernel y no permiten una gestión proactiva de contenedores. Este proyecto propone una solución integral: desarrollar un módulo de kernel en C que exponga métricas avanzadas de procesos y contenedores (CPU, memoria, E/S) a través de /proc, junto con un Daemon en GO que no solo presente estos datos de forma legible, sino que también automatice decisiones (como terminar contenedores que excedan umbrales de recursos) a su vez que guarda datos importantes en Valkey para su posterior uso en Grafana. Para validar el sistema, se implementará cronjobs que generen contenedores de prueba cada minuto, simulando condiciones de carga y permitiendo evaluar la eficacia de las acciones correctivas. Así, el proyecto combina aprendizaje en programación a bajo nivel (kernel) y alto nivel (GO), mientras resuelve un problema real en entornos contenerizados: la monitorización y estabilización autónoma del sistema. A su vez que de dichos datos se mostraran de manera visual por medio de un dashboard en Grafana.

1. **Un módulo de kernel desarrollado en C** que actuará como sensor de bajo nivel, accediendo directamente a las estructuras internas del kernel para capturar métricas detalladas tanto de los procesos asociados a contenedores como de los procesos generales del sistema, incluyendo consumo de recursos como CPU, memoria y E/S.

2. **Una Daemon en GO** que funcionará como cerebro del sistema, procesando los datos del kernel en tiempo real para:
    - Tomar decisiones autónomas detener y eliminar contenedores basadas en umbrales dinámicos y patrones de comportamiento establecidos previamente.

    - Simular y validar la eficacia del sistema bajo condiciones de uso prolongado.
    - Ejecutar scripts de automatización durante la ejecución.

3. **Un Cronjob** encargado de ejecutar el script que generara los contenedores de Docker cada minuto.

4. **Un Dashboard en Grafana** utilizado para mostrar la información que recolecta el servicio de GO.

5. **Una sonda eBPF** para la observabilidad profunda a nivel de sistema operativo enfocada en la terminación de procesos.

Actividad Extra (Puntos adicionales – No obligatoria)

Como actividad adicional y opcional, el estudiante podrá implementar y compilar un módulo de kernel desarrollado en Rust, con el objetivo de reforzar el entendimiento del desarrollo a bajo nivel utilizando lenguajes modernos soportados por el kernel de Linux.

1. Crear un módulo de kernel en **Rust** que, al ser cargado, muestre en el log del kernel el mensaje:

   ```
   Hola Mundo <#CARNET>
   ```

2. Compilar correctamente el módulo utilizando el soporte de Rust para el kernel de Linux.

3. Cargar y descargar el módulo en el sistema utilizando las herramientas correspondientes.

4. Verificar el correcto funcionamiento del módulo mediante la lectura de los mensajes del kernel.

MÓDULO DE KERNEL

Deberá crear un módulo que capture las métricas necesarias para el análisis de la memoria y los contenedores activos en el sistema.

La información debe ser capturada y guardada en la carpeta `/proc`:

1. Capturar en MB o KB (a discreción del estudiante):

- Total de memoria RAM
- Memoria RAM libre
- Memoria RAM en uso

2. Todos los procesos relacionados a los contenedores generados por el script así como los procesos generales del sistema deberán contar con:

- PID
- Nombre
- Línea de comando que se ejecutó o ID del contenedor
- VSZ (Tamaño de la memoria virtual en KB)
- RSS (Tamaño de memoria física en KB)
- Porcentaje de Memoria utilizada
- Porcentaje de CPU utilizado

Sugerencias:

- Utilizar la estructura task_struct (del kernel de Linux) para filtrar correctamente los procesos relacionados con los contenedores y extraer la información necesaria.
- En dado caso el porcentaje de CPU sea un número extremadamente grande se permite mantenerlo así debido a los cálculos diferenciales que retorna el kernel.

3. Los datos deberán ser guardados en sus respectivos archivos en /proc

- Módulo de Procesos de Contenedores: `/proc/continfo_pr2_so1_#CARNET`

CRONJOB

Se implementará un cronjob con ejecución cada **2 minutos** para simular una carga de trabajo variable. Su lógica consistirá en:

- Despliegue Aleatorio: Orquestar la creación de 5 contenedores, seleccionando aleatoriamente entre las imágenes de prueba desarrolladas.
- Contenedor Intruso: Imagen personalizada creada por el estudiante (Ataque de lectura a archivos sensibles).

IMÁGENES DE DOCKER (Carga de Trabajo)

Se desarrollarán 3 imágenes personalizadas para emular distintos perfiles de consumo:

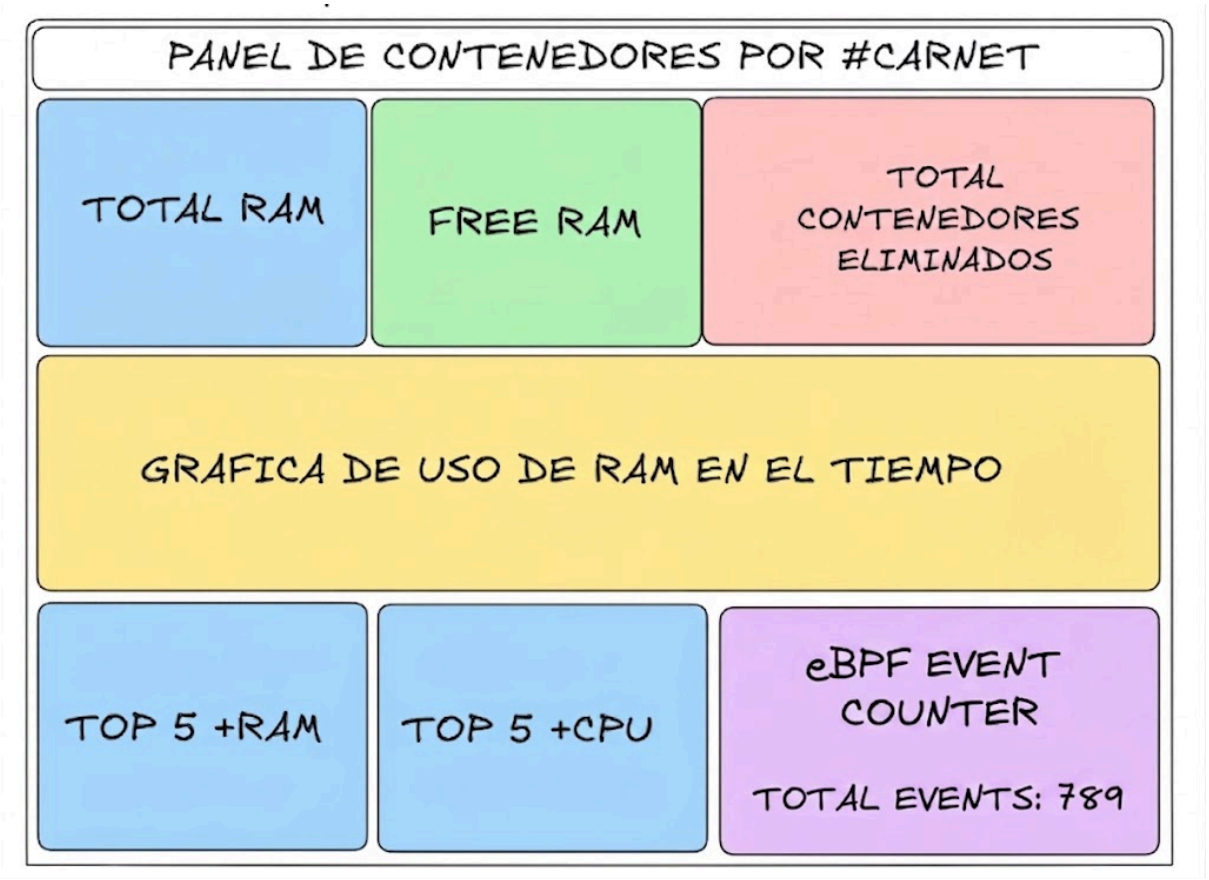
Categoría	Imagen Requerida	Descripción Funcional	Imagen Recomendada de Docker Hub	Comando de Ejemplo
Alto Consumo	go-client	Consumo significativo de memoria RAM.	roldyoran/go-client	docker run -d roldyoran/go-client
Alto Consumo	alpine	Alta carga computacional en la(s) CPU(s).	alpine	docker run -d alpine sh -c "while true; do echo '2^20' | bc > /dev/null; sleep 2; done"
Bajo Consumo	alpine	Consumo ínfimo de RAM y CPU.	alpine	docker run -d alpine sleep 240

GRAFANA

Como estudiante deberá realizar un dashboard en Grafana que muestre la siguiente información.

Métrica / Visualización	**Descripción y Detalles**	**Tipo de Visualización Sugerido**
Visión General del Sistema		
Total de RAM	[Dato en Texto]	Tarjeta (Card) o Indicador
RAM Usada	[Dato en Texto]	Tarjeta (Card) o Indicador
Memoria Libre	[Dato en Texto]	Tarjeta (Card) o Indicador
Evolución Temporal		
Uso de RAM a lo largo del tiempo	Monitorización del consumo de RAM con línea de evolución. Eje X: timestamp.	Gráfico de Líneas (Time Series) o Tarjeta (Card) o Indicador
Contenedores Eliminados a lo largo del tiempo	Número de contenedores eliminados en cada momento. Eje X: timestamp.	Gráfico de Barras (Time Series) o Tarjeta (Card) o Indicador
Top Rankings (Consumos Históricos)	**Nota: Incluye contenedores activos e inactivos/eliminados.**	
Top 5 Contenedores por Consumo de RAM	Muestra el PID y ID del proceso/contenedor.	Gráfico de Pastel (Pie Chart)
Top 5 Contenedores por Consumo de CPU	Muestra el PID y ID del proceso/contenedor.	Gráfico de Pastel (Pie Chart)
Eventos eBPF	Panel que muestre un contador de los contenedores eliminados exitosamente según capturas de sys_kill.	Contador o tabla de información

Dashboard a Realizar:



OBSERVABILIDAD CON eBPF

El programa eBPF estará dedicado exclusivamente a auditar las acciones de eliminación que realiza el Daemon en Go.

Llamada al sistema a interceptar: El programa eBPF debe adjuntarse a la llamada al sistema sys_kill o similar (utilizada para enviar señales de terminación a los procesos).

Cada vez que el Daemon envíe la instrucción de matar a un contenedor, eBPF debe capturar:

- PID del proceso emisor (opcional)
- PID del proceso receptor (el contenedor) (opcional)
- Señal enviada (ej. SIGTERM/SIGKILL o similar).

Flujo de datos: Esta información interceptada por eBPF debe ser enviada al Daemon de Go. El Daemon usará este evento capturado por eBPF como la "confirmación real" a nivel de kernel de que el contenedor fue terminado, y solo en ese momento registrará el evento de "Contenedor Eliminado" en Valkey.

Daemon de GO

Descripción (corazón del proyecto):
Se requiere desarrollar un **gestor de contenedores en Go** encargado del análisis, ejecución y comunicación entre los diferentes componentes del servicio.

Debe garantizar **manejo seguro de memoria** y cumplir con las siguientes funcionalidades:

1. Inicio del servicio

- Crear un contenedor de **Grafana** al inicializar el código.
- Grafana será el encargado de leer los logs generados por el servicio de Go después del análisis de los datos.
- Se recomienda utilizar un Docker Compose para que el contenedor se pueda comunicar con la base de datos Valkey

2. Cronjob

- El daemon de Go iniciará la implementación y ejecución del cronjob en el sistema operativo, lo que activará el proceso de creación de contenedores.

3. Ejecución del script para cargar el módulo de kernel

- El daemon de Go ejecutará un script encargado de cargar e inicializar el módulo de kernel.

4. Loop principal (ejecución cada 20 a 60 segundos)

- El daemon operará de manera infinita en segundo plano.
- En cada iteración (cada 20 a 60 segundos), realiza lo siguiente:
    - Lectura del archivo en `/proc/continfo_pr2_so1_#CARNET`
    - Deserialización del contenido
    - Análisis para la gestión de contenedores (alta/baja, ajuste de recursos)
        - detener y eliminar los contenedores según cálculos explicados en la siguiente sección.
    - Generación y almacenamiento de registros (logs) en una base de datos Valkey, diseñado para su posterior visualización en Grafana

5. Finalización del servicio

- Antes de finalizar, el servicio deberá **eliminar el cronjob** asociado a la creación de los contenedores para evitar una sobrecarga en el sistema.

RESTRICCIONES CLAVE

Contenedores en la máquina

- Siempre deben existir **3 contenedores de bajo consumo** en todo momento.
- Siempre deben existir **2 contenedores de alto consumo** en todo momento.
- **No debe eliminarse el contenedor de Grafana.**

Análisis y ordenamiento de contenedores

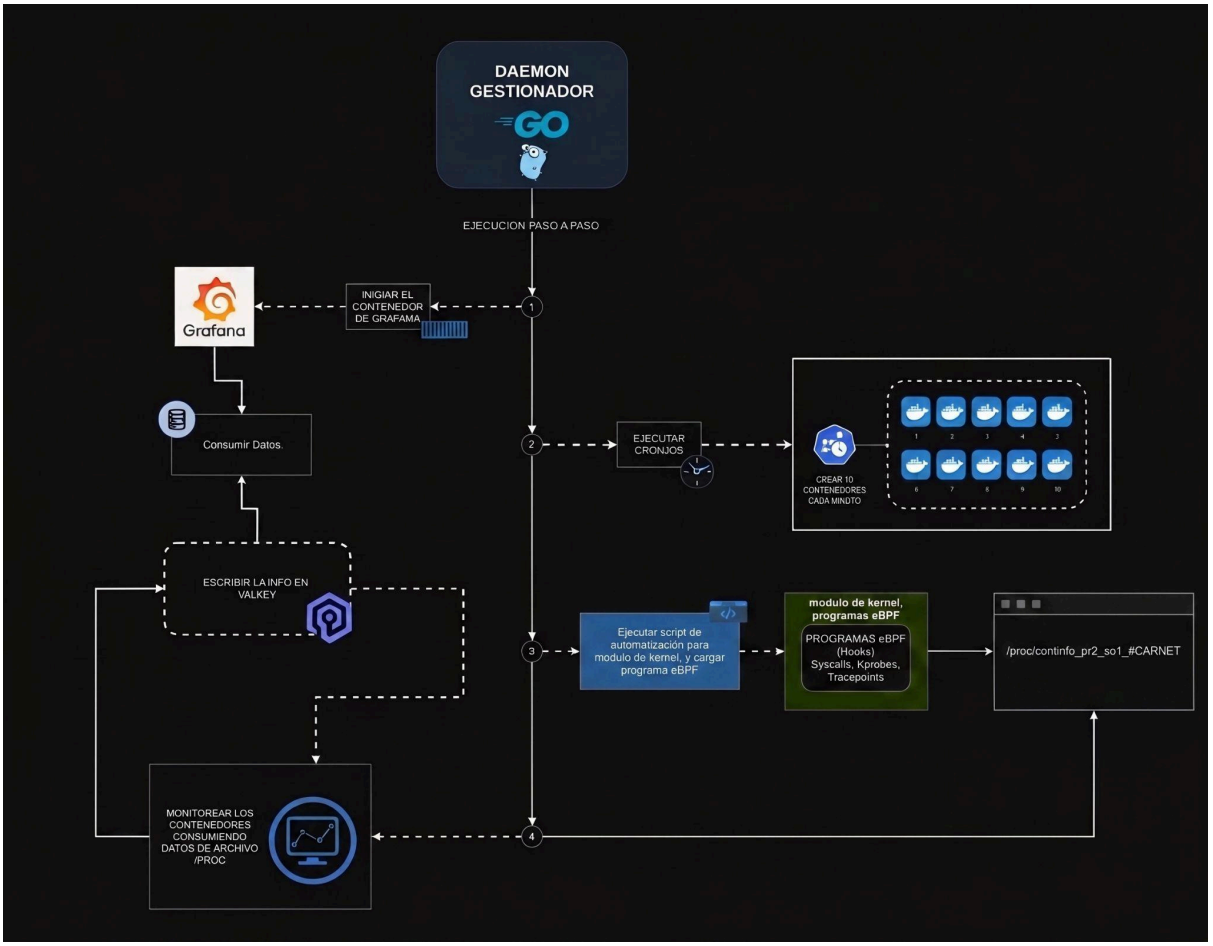
Durante la gestión de contenedores, aplicar ordenamientos basados en:

- Uso de RAM
- VSZ (Tamaño de memoria virtual en KB)
- RSS (Memoria física residente en KB)
- Uso de CPU

Toma de decisiones

El servicio debe decidir **qué contenedores eliminar y cuáles mantener**, considerando las instrucciones dadas anteriormente.

ARQUITECTURA



3.3 Entregables

1. Código fuente en repositorio privado GitHub nombre
   UTILIZAR MISMO REPOSITORIO que en el Proyecto 1, y crear una carpeta para el Proyecto 2 llamada proyecto2
   Deberá añadir a los auxiliares como colaboradores
   Usuarios de auxiliares: @JoseLorenzana272, @KINGR0X

2. Manual técnico y guía de instalación
3. Capturas de prueba y evidencia funcional

4. Material de apoyo

Link Repositorio del curso

5. Metodología

1. Código fuente en repositorio privado GitHub nombre
   Carnet#_LAB_SO1_1S2026
   Deberá añadir al auxiliar como colaborador
   Usuarios de auxiliares: @JoseLorenzana272, @KINGR0X

2. Manual técnico y guía de instalación
3. Capturas de prueba y evidencia funcional

6. Recursos y herramientas a utilizar

Listado de materiales que los estudiantes deberán usar o investigar:

-

7. Desarrollo de Habilidades Blandas

- Gestión de tiempo personal
- Responsabilidad sobre el diseño e implementación
- Resolución autónoma de problemas
- Autoevaluación y reflexión sobre el trabajo realizado

8. Cronograma

El cronograma describe las etapas clave del proyecto, los plazos estimados para cada una, y el proceso de asignación, elaboración y calificación de las tareas. Los estudiantes deberán seguir este plan para asegurar que el proyecto avance de manera organizada y cumpla con los plazos establecidos. Cada fase incluye la asignación de tareas, el tiempo estimado para su elaboración, y el momento de su calificación.

Tipo	Fecha Inicio	Fecha Fin
Asignación de Proyecto	20/08/2026	20/08/2026
Elaboración	20/08/2026	17/09/2026
Calificación	18/09/2026	19/09/2026

9. Rúbrica de Calificación

9.1 Requisitos para optar a la calificación

Antes de la evaluación del proyecto, los estudiantes deben cumplir con los requisitos que se indiquen en esta sección.

Tema	Descripción	Cumple (Si/No)
Entrega Completa	Se entregan todos los archivos solicitados (código fuente .c, Makefile, informe técnico).	
Funcionalidad Mínima de Listado	Al leer el archivo en /proc, se muestra un listado de procesos que incluye al menos PID, Nombre del Comando, Memoria, CPU para varios contenedores activos.	

8.2 Resumen de Puntuaciones

Área	Puntos Totales	Puntos Obtenidos
1. Conocimiento técnico (85%)		
Funcionalidad del Sistema (Módulo + Daemon GO)	30	
Extracción (Kernel) y Parseo (GO) de Datos	20	
Observabilidad con eBPF	20	
Calidad del Código y Build Systems	15	
Sub-Total Conocimiento	**85**	
2. Habilidades y documentación (15%)		
Informe Técnico y Claridad	5	
Comprensión (Defensa/Preguntas)	10	
Sub-Total Habilidades	**15**	
TOTAL	**100**	

8.3 Detalle de la Calificación

Criterio	Descripción	Puntos Máximos	Puntuación Obtenida
1. Funcionalidad del Sistema (Módulo + CLI)		**40**	
1.1 Compilación y Enlazado Módulo Kernel	`Makefile` correcto, módulo compila sin errores.	8	
1.2 Carga y Descarga del Módulo Kernel	Módulo se carga/descarga limpiamente (`insmod`, `rmmod`).	8	
1.3 Creación y Acceso a Interfaz `/proc`	Archivo en `/proc` se crea/elimina y es legible.	8	
1.4 Compilación y Ejecución del Daemon GO	Daemon GO compila y se ejecuta.	8	
1.5 Interacción con /proc y Base de Datos Valkey	El daemon GO interactúa con /proc y DB Valkey correctamente.	8	
2 Correcta Extracción (Kernel) y Parseo de Datos		**20**	
2.1 Iteración de lista de procesos (Kernel)	Módulo itera correctamente `task_struct`.	7	
2.2 Extracción de Información (PID, Memoria, CPU) (Kernel)	Se extraen y muestran correctamente PID, `Memoria, CPU,` en `/proc`.	8	
2.3 Parseo de Datos en GO	El Daemon de GO parsea correctamente la información del	5	

	archivo en `/proc` para ingresarla en una DB Valkey		
3. Observabilidad con eBPF		**25**	
3.1 Intercepción de syscall (sys_kill) y Captura de Eventos	El programa eBPF se adjunta correctamente a sys_kill y captura los PID y señales emitidas con precisión.	12	
3.2 Comunicación de eventos eBPF a Daemon GO	La información es enviada e interpretada por GO como confirmación real de terminación y se registra en Valkey.	13	
Sub-Total de Puntos		**85**	
2.Habilidades y Documentación			
Manual Tecnico y Manual de Usuario	Informe claro, conciso, explica diseño, implementación, uso de ambos componentes.	**10**	
Comprensión Demostrada (Defensa/Preguntas)	Estudiante demuestra comprensión de conceptos kernel, GO, su código y decisiones.	**5**	
Sub-Total de Puntos		**15**	
Total		**100**	

8.5 Comentarios Generales

Seguir utilizando el mismo repositorio creado en el proyecto 1, solo que ahora colocando todo el contenido de este proyecto en una carpeta llamada ‘proyecto2’

Detalle de la Calificación

	Criterio/Nivel			Nivel de evaluación	
No.	**Criterio de evaluación**	**Punteo máximo**	**Satisfactorio (100% - 61%)**	**Necesita mejorar (60% - 0%)**	**Punteo Obtenido**
1	**Funcionalidad del Sistema (Módulo Kernel + Daemon GO)**	**40**			
1.1	Compilación y enlazado del módulo Kernel	8	**Descripción:** Makefile correcto, el módulo compila sin errores.**Rango del punteo:** 8-5	**Descripción:** El módulo no compila o presenta errores.**Rango del punteo:** 4–0	
1.2	Carga y descarga del módulo Kernel	8	**Descripción:** El módulo se carga y descarga correctamente (`insmod`, `rmmod`).**Rango del punteo:** 8-5	**Descripción:** El módulo no carga o no se descarga correctamente.**Rango del punteo:** 4–0	
1.3	Creación y acceso a la interfaz `/proc`	8	**Descripción:** El archivo en `/proc` se crea, elimina y es legible.**Rango del punteo:** 8-5	**Descripción:** El archivo no existe o no es legible.**Rango del punteo:** 4–0	

1.4	Compilación y ejecución del Daemon GO	8	**Descripción:** El daemon GO compila y se ejecuta correctamente.**Rango del punteo:** 8-5	**Descripción:** El daemon no compila o no se ejecuta.**Rango del punteo:** 4–0	
1.5	Interacción con /proc y Base de Datos Valkey	8	**Descripción:** El daemon GO interactúa con /proc y DB Valkey correctamente. **Rango del punteo:** 8-5	**Descripción:** Falla al interactuar con /proc o no guarda datos en Valkey. **Rango del punteo:** 4–0	
	Sub-Total de Puntos	**40**			**40**
2	**Correcta Extracción (Kernel) y Parseo de Datos**	**20**			
2.1	**Iteración de lista de procesos (Kernel)**	**7**	**Descripción:** Itera correctamente la estructura `task_struct`. **Rango del punteo: 7-4**	**Descripción:** Iteración incorrecta o incompleta. **Rango del punteo: 4-0**	
2.2	**Extracción de información (PID, Memoria, CPU)**	**8**	**Descripción:** Se extraen y muestran correctamente PID, memoria y CPU.	**Descripción: Información incompleta o incorrecta. Rango del punteo: 4-0**	

			Rango del punteo: 8-5		
2.3	**Parseo de datos en GO**	**5**	**Descripción:** El daemon GO parsea correctamente la información y la almacena en Valkey. **Rango del punteo: 5-4**	**Descripción:** Error en el parseo o almacenamiento. **Rango del punteo: 3-0**	
	Sub-Total de Puntos	**20**			**20**
3	**Observabilidad con eBPF**	**25**			
3.1	**Intercepción de syscall (sys_kill) y Captura de Eventos**	**12**	**Descripción:** El programa eBPF se adjunta correctamente a sys_kill y captura los PID y señales emitidas con precisión. **Rango del punteo: 10-6**	**Descripción:** El programa eBPF no se adjunta correctamente, no captura los datos o presenta errores de lógica al interceptar. **Rango del punteo: 5-0**	
3.2	**Comunicación de eventos eBPF a Daemon GO**	**13**	**Descripción:** La información es enviada e interpretada por GO	**Descripción:**No hay comunicación efectiva con GO, o el registro de confirmación no se maneja	

			como confirmación real de terminación y se registra en Valkey. **Rango del punteo: 6–10**	adecuadamente en Valkey. **Rango del punteo: 0–5**	
	Sub-Total de Puntos	**25**			**25**
4	**Habilidades y Documentación**	**15**			
4.1	Manual Técnico y Manual de Usuario	**10**	**Descripción:** Informe claro, conciso y completo. **Rango del punteo: 6–10**	**Descripción:** ocumentación incompleta o confusa. **Rango del punteo: 0–5**	
4.2	Comprensión demostrada (Defensa/Preguntas)	**5**	**Descripción: Explica y justifica correctamente las decisiones técnicas.Rango del punteo: 3–5**	**Descripción: No demuestra comprensión adecuada.Rango del punteo: 0–2**	
	Sub-Total de Puntos	**15**			**15**
	TOTAL	**100**			**100**